In-Class Exercise/Demo

In-Class Demo

- In today's demo exercise, we will practice both OOP and VUEjs.
- But first, create a new github repository **m2-w4-d1-exercise** and clone this in the WESTCLIFF folder.
- Next, head to GAP Week 8 Day 1 and download the zip file **wk4-day1-exercise**, unzip and drop all the files into the cloned folder.
- Launch VS Code and make sure auto save is on.
- Navigate to open the cloned wk4-day1-exercise folder.

In-Class Demo

- Open **object.html** in VS Code.
- Preview the page on the browser.
- You should see a heading **JavaScript Objects**.
- Return to VS Code.
- Add **${car.type} ${car.model}** and **${car.price}** within the curly braces.
- Preview on the browser console. You should see this message printed:

Honda has many models but Civic is $25000

In-Class Demo

- Open **object-bracket.html** in VS Code.
- Preview the page on the browser console.
- You should see **WestCliff Bootcamp**.
- Return to VS Code.
- Add *.split(' ')[0]* ie: **${school.name.split(' ')[0]}**
- Preview on the browser console again. The message printed is updated to just:

WestCliff

In-Class Demo

- Open **constructors.html** in VS Code
- Preview the page on the browser console.
- You should see **undefined**. Return to VS Code.
- Update car1's function calling by passing these arguments **'Focus', 'Ford'** as *name* and *maker* respectively
- Update car2's function calling by passing these arguments **'Elantra', 'Hyundai'** as *name* and *maker* respectively
- Preview on the browser console again. The message printed is updated to:

  Focus

  Elantra

In-Class Demo

- Open **inheritance.html** in VS Code.
- Preview the page on the browser console.
- You should see **false** printed. Return to VS Code.
- The ***hasOwnProperty()*** method checks to see if a property exist. In this case, there is none, so it return false.
- Now let's pass **property1** to it, ie: **hasOwnProperty('property1')**
- Preview on the browser console again. The message printed is updated to:

  True
- This means **Property1** exists in the object

In-Class Demo

- Open **encapsulation.html** in VS Code.
- Review the code and preview the page on the browser console.
- You should see **Area is, Radius is, Area is** printed. Return to VS Code.
- For Technique 1:
  - On the variable circle1, pass 3 as radius to the createCircle() function
  - Then write the following **console.log** statements for output:

```
console.log("Area is " + circle1.area());
console.log("Radius is " + circle1.radius); // THIS WILL FAIL
```

continue on next slide

In-Class Demo

- Preview the page on the browser console again.
- You should now see this printed on the console:

```
Area is 28.274333882308138
Radius is undefined
```

- Return to VS Code.
- Next, we will move on to Technique 2. On this technique, notice there's a second function *circumference* and a *diameter* variable declared within the *createCircle* function scope.

continue on next slide

In-Class Demo

- Let's add the following codes:
  - On the variable circle2, pass 3 as radius to the createCircle() function
  - Then write the following **console.log** statements for output:

```
console.log("Circumference is " + circle2.circumference());
console.log("Diameter is " + circle2.diameter); // THIS WILL FAIL
```

- Preview the page on the browser console again.
- You should now see this printed on the console:

```
Circumference is 18.84955592153876
Diameter is undefined
```

In-Class Demo

- Open **polymorphism.html** in VS Code.
- Review the code and preview the page on the browser console.
- There's nothing printed to the console at this time.
- Let's pass a number as argument to the calling:

```
dog.run(15);
```

- And let's pass a string as argument to the next calling:

```
dog.run('20 Miles/hr');
```

- Preview the page on the browser console again. This should be printed:

```
The dog runs at a speed of 15 Miles/hr
The dog runs at a speed of 20 Miles/hr
```

In-Class Demo

- Let's add a second method:

```
DogType.eat = function(food) {
  if(food == 'dry') {
    console.log('The dog eats ' + food + ' food');
  } else if(food == 'wet') {
    console.log('The dog eats ' + food + ' food');
  }
}
```

- Next, we'll apply the method to our object and pass an argument to the calling:

```
//Apply method 2 to dog
dog.eat('dry'); //The dog eats dry food
```

- Preview the page on the browser console again. This should be printed:

```
The dog eats dry food
```

In-Class Demo

- We'll practice JavaScript class next.
- Open **classes1.html** in VS Code.
- In here, we have a class/function with some parameters/properties set to some values.
- Let's create objects from this class, while also set some new values to the parameters/properties:

```
//Create object 1
var person1 = new Person();
person1.firstName = "Steve";
person1.lastName = "Jobs";

alert(person1.firstName + " " + person1.lastName);
```

continue on next slide

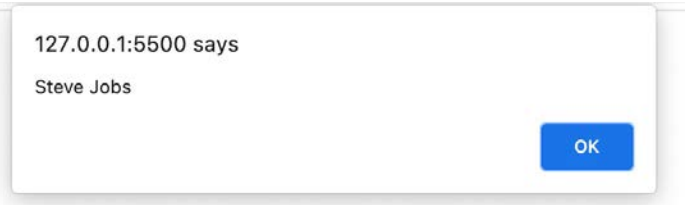
In-Class Demo

- And a second object:

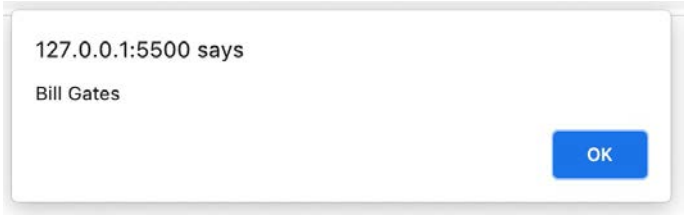
```
//Create object 2
var person2 = new Person();
person2.firstName = "Bill";
person2.lastName = "Gates";

alert(person2.firstName + " " + person2.lastName );
```

- Preview the page on the browser:



- Follow by:



In-Class Demo

- Open **classes2.html** in VS Code.
- In here, we have a class/function with some parameters/properties set to some values. We'll add a method within the class:

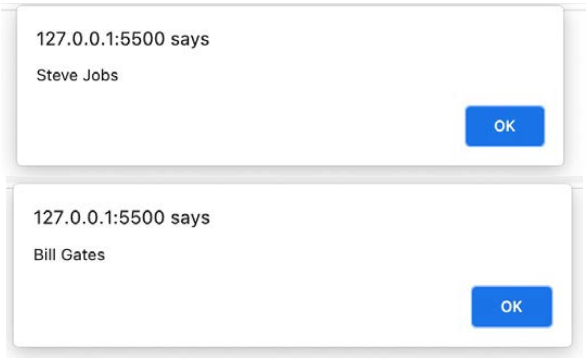
```
//Create a method
this.getFullName = function(){
    return this.firstName + " " + this.lastName;
}
```

- Let's apply this method to our existing objects:

```
//apply method
alert(person1.getFullName());
```

```
//apply method
alert(person2.getFullName());
```

When preview on the browser:



In-Class Demo

- Open **classes3.html** in VS Code.
- In here, we have a class/function with the existing method but no parameters/properties set at this time. We'll now add that as a constructor:

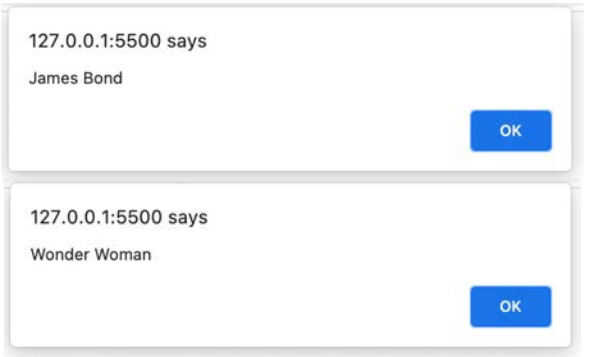
```
//Parameters/Properties as Constructor
this.firstName = FirstName || "unknown"; //default value
this.lastName = LastName || "unknown"; //default value
this.age = Age || 25; //default value
```

- Let's pass values to our existing objects:

```
//Create object 1
var person1 = new Person("James","Bond",50);
alert(person1.getFullName());
```

```
//Create object 2
var person2 = new Person("Wonder","Woman");
alert(person2.getFullName());
```

When preview on the browser:



In-Class Demo

- Open **vue_demo1.html**.
- Let's create a simple Hello World application using VUE.
- Notice in the file, the following has been setup:
  - A div element with an id value
  - A script element
- First, we will create and instantiate a vue instance within the <script> element:

```
<script>

    var app = new Vue({
        el: '#vueapp'
    });

</script>
```

In-Class Demo

- Next, add a simple data in the instance:

```
var app = new Vue({
    el: '#vueapp',
    data: {
        greeting: 'Hello World!'
    }
});
```

- Then call the data to be displayed in the div element:

```
<div id="vueapp">
    {{ greeting }}
</div>
```

Previewing on the browser:

Hello World!

In-Class Demo

- In the next exercise, we'll add a second data in the instance. Open **vue_demo2.html** in VS Code.

```
var app = new Vue({
    el: '#vueapp',
    data: {
        greeting: 'Hello World!',
        message: "I'm learning VUE for the first time!"
    }
});
```

- Then call the additional data to be displayed in the div element:

```
<div id="vueapp">
    {{ greeting }}
    {{ message }}
</div>
```

Previewing on the browser:

Hello World! I'm learning VUE for the first time!

In-Class Demo

- In the next exercise, we'll add a third data that contain some HTML markup in the instance. This data template that will be added to the browser DOM. Open **vue_demo3.html** in VS Code.

```
var app = new Vue({
    el: '#vueapp',
    data: {
        greeting: 'Hello World!',
        message: "I'm learning VUE for the first time!",
        htmlcontent : "<h1>Vue Js Template</h1>"

    }
});
```

- Then call the data template to be displayed in the div element:

```
<div id="vueapp">
    {{ greeting }}
    {{ message }}
    <div v-html = "htmlcontent"></div>
</div>
```

Previewing on the browser:

Hello World! I'm learning VUE for the first time!

Vue Js Template

In-Class Demo

- Next, we'll practice VUE components. We will start with a simple global registered component. Open **vue_demo4.html** in VS Code. Create a new component with a template like so:

```
<script>
    Vue.component('mycomponent1',{
        template : '<div><h1>Welcome to VUE Components!</h1></div>'
    });
```

- Next, we'll instantiate a new instance:

```
    Vue.component('mycomponent1',{
        template : '<div><h1>Welcome to VUE Components!</h1></div>'
    });

    var vm1 = new Vue({
        el: '#component1'
    });
</script>
```

In-Class Demo

- Then call the component as custom HTML element within the div element:

```
<div id = "component1">
    <mycomponent1></mycomponent1>
</div>
```

Previewing on the browser:

Welcome to VUE Components!

In-Class Demo

- We will add a second global registered component. Open **vue_demo5.html** in VS Code and add the following scripts below the first component:

```
Vue.component('mycomponent1',{
    template : '<div><h1>Welcome to VUE Components!</h1></div>'
});
Vue.component('mycomponent2',{
    template : '<div><p>VUE is interesting!</p></div>'
});
```

- Next, we'll instantiate a second instance:

```
var vm1 = new Vue({
    el: '#component1'
});
var vm2 = new Vue({
    el: '#component2'
});
```

In-Class Demo

- Then call the second component as custom HTML element within the div element:

```
<div id = "component1">
    <mycomponent1></mycomponent1>
</div>
<div id = "component2">
    <mycomponent2></mycomponent2>
</div>
```

Previewing on the browser:

Welcome to VUE Components!

VUE is interesting!

In-Class Demo

- Let's look at an example of sharing component. We'll instantiate a third instance but maintain with two components. Open **vue_demo6.html** in VS Code.

```
    var vm2 = new Vue({
        el: '#component2'
    });
    var vm3 = new Vue({
        el: '#component3'
    });
</script>
```

- Then add a third div element and call the second component :

```
<div id = "component2">
   <mycomponent2></mycomponent2>
</div>
<div id = "component3">
   <mycomponent2></mycomponent2>
</div>
```

Previewing on the browser:

Welcome to VUE Components!

VUE is interesting!

VUE is interesting!

The 3rd element shares content from the 2nd component.

In-Class Demo

- In the next example, we will turn the two components to local registered components by moving them into the first two instances respectively. Open **vue_demo7.html** in VS Code. We'll start with the first component:

```
var vm1 = new Vue({
    el: '#component1',
    components:{
        'mycomponent1': {
            template : '<div><h1>Welcome to VUE Components!</h1></div>'
        }
    }
});
```

- Next we will put the second component in the second instance:

```
var vm2 = new Vue({
    el: '#component2',
    components:{
        'mycomponent2': {
            template : '<div><p>VUE is interesting!</p></div>'
        }
    }
});
```

In-Class Demo

Previewing on the browser:

Because the components are local registered in the instance, the third instance will not have access to the component.

In-Class Demo

Due:

Today 10.30PM PT

Submission:

Post Github URL link in the dropbox at GAP Week 8 Day 1 Exercise